ECS Task

1. Setup a highly available ecs cluster with load balancer and dynamic port mapping.
NOTE:
Your cluster should maintain atleast 6 tasks at any point of time and should be highly availble across multiple AZ's.
Use the below image from deployment:
`sabair0509/hiring-app:works`

AWS ECS Task — Highly Available ECS Cluster with ALB and Dynamic Port Mapping

1. Task requirement

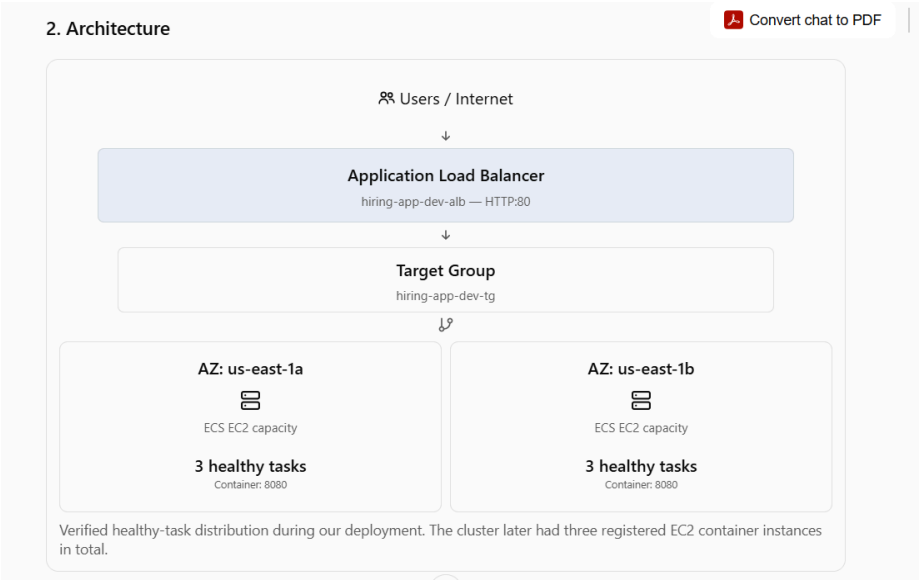
> Set up a highly available ECS cluster with an Application Load Balancer and dynamic port mapping. The cluster should maintain at least six tasks and distribute them across multiple Availability Zones.

Docker image:

sabair0509/hiring-app:works

Region: `us-east-1` (N. Virginia)

Services used: Amazon ECS, EC2, Auto Scaling Group, IAM, VPC, Security Groups, Application Load Balancer, Target Group and CloudWatch Logs.



3. Implementation steps

Step 1 — Verify AWS configuration

We used our existing Ubuntu EC2 server, `Docker-01-Server`, to run AWS CLI commands.

aws sts get-caller-identity

Purpose: verify the AWS account and IAM role.

```
aws ec2 describe-vpcs \
  --region us-east-1 \
  --query 'Vpcs[*].[VpcId,IsDefault,CidrBlock]' \
  --output table
```

We selected the default VPC:

vpc-09ace864d17ee9449
CIDR: 172.31.0.0/16

We also listed the subnets:

```
aws ec2 describe-subnets \
  --region us-east-1 \
  --query 'Subnets[*].[SubnetId,AvailabilityZone,VpcId]' \
  --output table
```

Challenge faced: The `Docker-ECR-Role` IAM role did not have permission to run `ecs:DescribeClusters`. Therefore, we continued the deployment using the AWS Console.

Step 2 — Create ECS Cluster

Go to:

AWS Console → ECS → Clusters → Create cluster

Configuration:

Field	Value
Cluster name	`hiring-app-dev-cluster`
Infrastructure	Fargate and Self-managed instances
Provisioning model	On-demand
ECS AMI	Amazon Linux 2023
EC2 instance type	`c7i-flex.large` (after troubleshooting)
Minimum instances	2
Maximum instances	6
Desired instances	2

VPC	Default VPC

We initially tried `t3.medium`, but EC2 Auto Scaling failed because the instance type was not eligible under our account's Free Tier restrictions.

We created a newer EC2 Launch Template version using `c7i-flex.large` and updated the Auto Scaling Group.

Result: ECS cluster created successfully, with two EC2 instances initially registered across two AZs.

Step 3 — Configure networking

For the ALB, we used public subnets in two different Availability Zones:

AZ	Public subnet
us-east-1a	`subnet-00444f9336cffeedf`
us-east-1b	`subnet-07efbd5ec50988192`

Both public subnets had routes to an Internet Gateway.

Step 4 — Create Security Groups

We configured two security groups.

ALB Security Group

Name: `hiring-app-alb-sg`

ID: `sg-0338de7f12679dac4`

Type	Port	Source
HTTP	80	`0.0.0.0/0`

ECS Instance Security Group

Name: `hiring-app-ecs-sg`

ID: `sg-0906a6c96ccf4834f`

Type	Ports	Source
Custom TCP	32768–65535	`sg-0338de7f12679dac4`

Purpose: allow ALB traffic to dynamically allocated container host ports without opening those ports to everyone.

Step 5 — Create ECS Task Definition

Navigate:

ECS → Task definitions → Create new task definition

Setting	Value
Family	`hiring-app-dev-task`
Revision	1

Launch environment	Amazon EC2
Operating system	Linux/X86_64
Network mode	`bridge`
Task CPU	0.25 vCPU
Task memory	1 GiB
Execution role	`ecsTaskExecutionRole`

Container configuration:

Field	**Value**
Container name	`hiring-app-container`
Docker image	`sabair0509/hiring-app:works`
Container port	8080
Host port	0 / dynamic

Protocol	TCP
Essential	Yes

CloudWatch Logs configuration:

Log group: /ecs/hiring-app-dev-task
Region: us-east-1
Stream prefix: ecs

Why bridge networking?

Bridge mode allows ECS to run multiple containers on the same EC2 instance using different host ports.

For example:

EC2 Host Port 32768 → Container Port 8080
EC2 Host Port 32769 → Container Port 8080
EC2 Host Port 32770 → Container Port 8080

This is called Dynamic Port Mapping.

Step 6 — Test the Task Definition

Before creating the ECS Service, we manually ran one task.

Navigate:

ECS → Clusters → hiring-app-dev-cluster → Run task

Select:

- Task Definition: `hiring-app-dev-task:1`
- Compute: EC2 Capacity Provider
- Desired tasks: 1

The task started successfully.

In the container's Network bindings, we verified that ECS mapped container port `8080` to host port `32768`.

After testing, we stopped this standalone task when pausing the lab overnight.

Step 7 — Create Target Group

Navigate:

EC2 → Target Groups → Create target group

Configuration:

Field	Value
Name	`hiring-app-dev-tg`
Target type	Instances
Protocol	HTTP
Default port	8080
VPC	`vpc-09ace864d17ee9449`
Health check protocol	HTTP
Health check path	`/hiring/`

We did not manually register the ECS instances with a fixed port.

ECS automatically registered the correct instance IDs and dynamically assigned ports when the Service was deployed.

Step 8 — Create Application Load Balancer

Navigate:

EC2 → Load Balancers → Create Application Load Balancer

Configuration:

Field	Value
Name	`hiring-app-dev-alb`
Scheme	Internet-facing
IP type	IPv4
VPC	Default VPC
Availability Zones	us-east-1a and us-east-1b
Security Group	`hiring-app-alb-sg`
Listener	HTTP:80
Target Group	`hiring-app-dev-tg`

Result: ALB created and status changed to ACTIVE.

Step 9 — Create ECS Service

Navigate:

ECS → Clusters → hiring-app-dev-cluster → Services → Create

Configuration:

Field	Value
Service name	`hiring-app-dev-task-service`
Task definition	`hiring-app-dev-task:1`
Environment	Amazon EC2
Capacity provider	Existing ASG Capacity Provider
Base	0
Weight	1
Scheduling strategy	Replica
Desired tasks	6
AZ rebalancing	Enabled

Load balancing:

Field	Value
Load balancing	Enabled
ALB	`hiring-app-dev-alb`
Existing listener	HTTP:80
Existing target group	`hiring-app-dev-tg`
Container	`hiring-app-container :8080`

Task placement:

AZ balanced spread

Underlying strategy:

spread(attribute:ecs.availability-zone)
spread(instanceId)

This distributes tasks across Availability Zones and then across instances.

We clicked Create Service.

Step 10 — Verify deployment

Initially, the ECS Service showed zero running tasks while the deployment was starting.

Later, ECS scaled to three registered EC2 instances and successfully deployed the six Hiring App tasks.

Final service status:

Metric	Result
Desired tasks	6
Running tasks	6
Pending tasks	0
Deployment	Success
Service state	Steady state

Step 11 — Verify Target Group Health

Navigate:

EC2 → Target Groups → hiring-app-dev-tg → Targets

We observed eight registered target entries, consisting of:

- Six healthy targets
- Two draining targets from earlier replacements
- Zero unhealthy targets

Healthy target distribution:

AZ	Healthy targets

us-east-1a	3
us-east-1b	3
Total	6

Observed host ports included `32768`, `32769` and `32770`, confirming dynamic port mapping.

Step 12 — Test Application URL

We opened the ALB URL in the browser:

Hiring App — ALB URL

The browser successfully displayed:

> Welcome to Techie Horizon

This verified external HTTP access through the Application Load Balancer.

Step 13 — Perform Self-Healing Test

We tested ECS's ability to replace a failed or stopped task.

1. Opened ECS Cluster → Tasks.
2. Selected only one running ECS Service task.
3. Clicked Stop.
4. Observed ECS Service Events.
5. Waited for ECS to launch a replacement task.

Actual event sequence:

Time	AWS ECS action
07:14	Deregistered one target

07:14 Began draining connections

07:14 Started replacement task

07:14 Registered replacement target

07:15 Reached steady state

Final result: 6 running tasks, 0 pending tasks and successful recovery.

This proves task-level self-healing. It does not by itself prove zero interruption or survival of an entire Availability Zone failure while continuously maintaining six running tasks.

4. Troubleshooting and challenges

Issue	Cause / Resolution
AWS CLI AccessDeniedException	IAM role lacked ECS permissions; continued using Console
EC2 instances not launching	Original instance type rejected under Free Tier eligibility constraints
Launch Template issue	Created and selected the newer version with `c7i-flex.large`
No ECS container instances	Waited for ASG provisioning and ECS agent registration

Security group mismatch	Replaced default SG source with ALB SG source
Dynamic ports not accessible	Allowed TCP 32768–65535 from ALB SG
Initial unhealthy ECS tasks	ECS replaced unhealthy tasks automatically
Additional targets showing	Previous targets were draining during deployment
EC2 shutdown delayed overnight	Stopped the standalone running ECS task and waited for ASG lifecycle termination

5. Important AWS CLI verification commands

We used AWS Console for most ECS operations, but these commands are useful for future troubleshooting after the appropriate IAM permissions are granted.

```
# Check identity
aws sts get-caller-identity

# ECS cluster details
aws ecs describe-clusters \
  --clusters hiring-app-dev-cluster \
  --region us-east-1

# List running tasks
aws ecs list-tasks \
  --cluster hiring-app-dev-cluster \
  --desired-status RUNNING \
  --region us-east-1

# Describe ECS service
aws ecs describe-services \
  --cluster hiring-app-dev-cluster \
  --services hiring-app-dev-task-service \
  --region us-east-1
```

```
# List registered EC2 container instances
aws ecs list-container-instances \
  --cluster hiring-app-dev-cluster \
  --region us-east-1
```

These are reference commands, not a claim that every command was executed successfully during our lab.

Q1. What is Amazon ECS?

Amazon ECS is a managed container orchestration service used to deploy, manage and scale containerized applications.

Q2. What is the difference between a Task Definition and a Task?

A Task Definition is a blueprint describing container image, CPU, memory, networking and other settings. A Task is a running instance of that definition.

Q3. What is an ECS Service?

An ECS Service maintains a desired number of tasks. If a task stops, ECS can launch another to restore the desired count.

Q4. What is dynamic port mapping?

Dynamic port mapping assigns an available host port to a container port.

In our project, multiple containers listened on port 8080 internally, but ECS assigned different EC2 host ports.

Q5. Why did we use an Application Load Balancer?

To distribute incoming HTTP requests across healthy ECS tasks and provide a single public endpoint.

Q6. What is a Capacity Provider?

A Capacity Provider manages or selects the computing resources available to run ECS tasks. Our service used a Capacity Provider connected to an EC2 Auto Scaling Group.

Q7. What is AZ balanced spread?

It is a task placement strategy that attempts to distribute ECS tasks evenly across Availability Zones and instances to improve availability.

Q8. What happens when an ECS task stops unexpectedly?

When the task belongs to an ECS Service, the service scheduler attempts to create a replacement task to maintain the configured desired count.

Q9. What is the difference between an ALB Listener and a Target Group?

The Listener accepts incoming connections and applies routing rules. The Target Group contains the backend destinations and performs health checks.

Q10. How did you troubleshoot unhealthy ECS tasks?

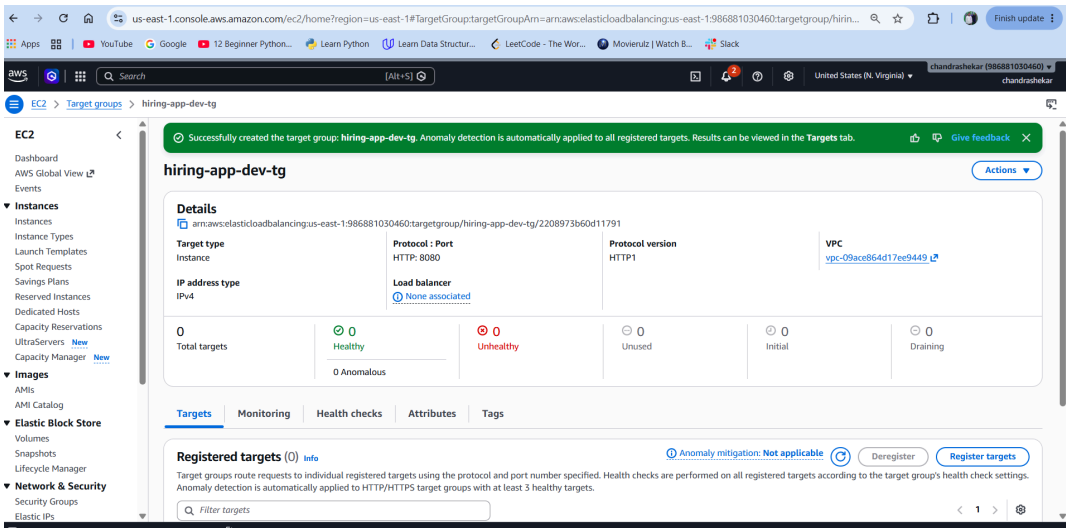
We checked ECS Service Events, task status, target health, dynamic port configuration and security group rules. We confirmed the application endpoint and observed ECS replacing unhealthy tasks.

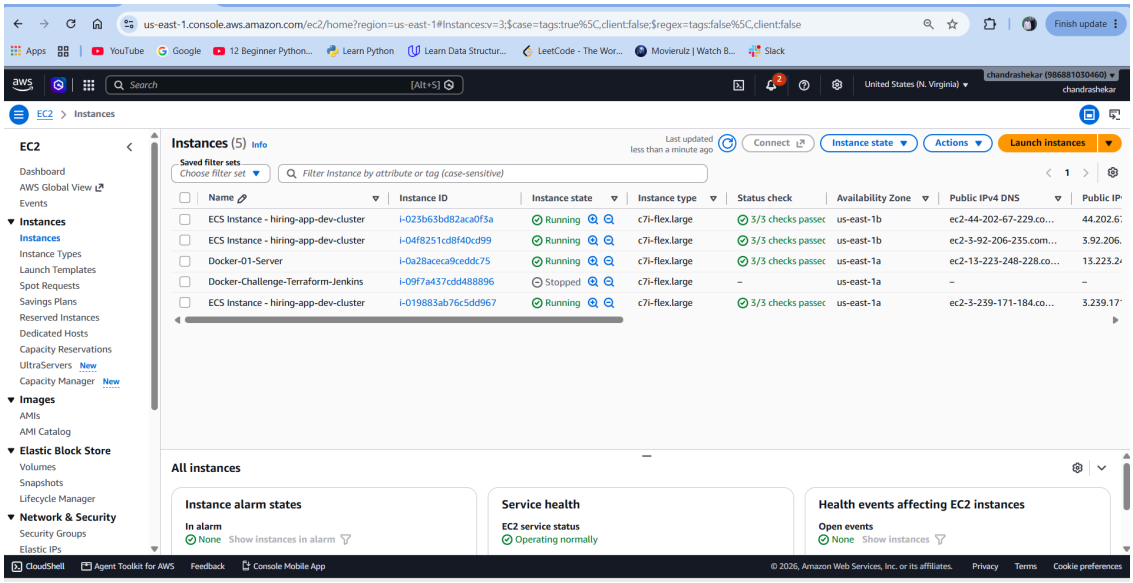
Q11. Can two ECS tasks run on one EC2 instance?

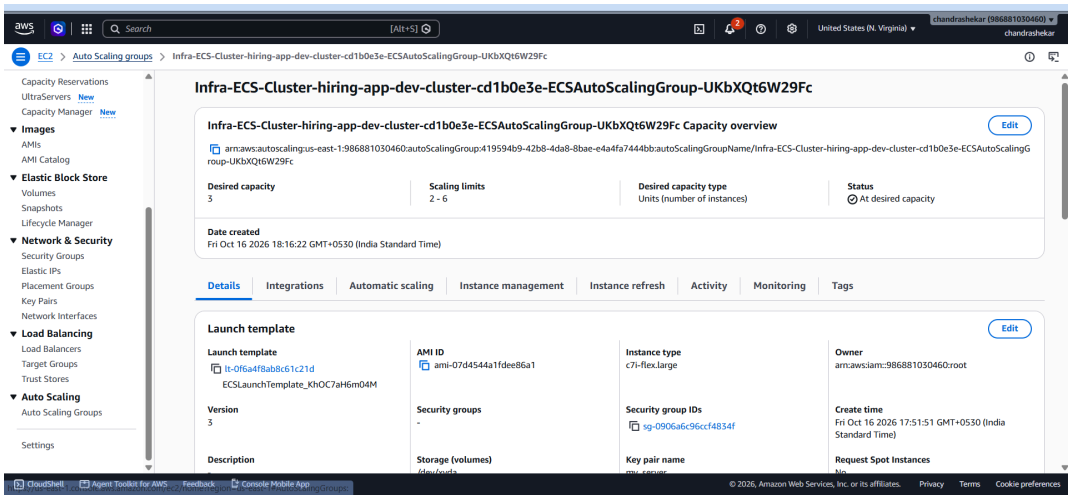
Yes. Multiple ECS tasks can run on one EC2 instance as long as adequate CPU, memory, networking and port capacity are available.

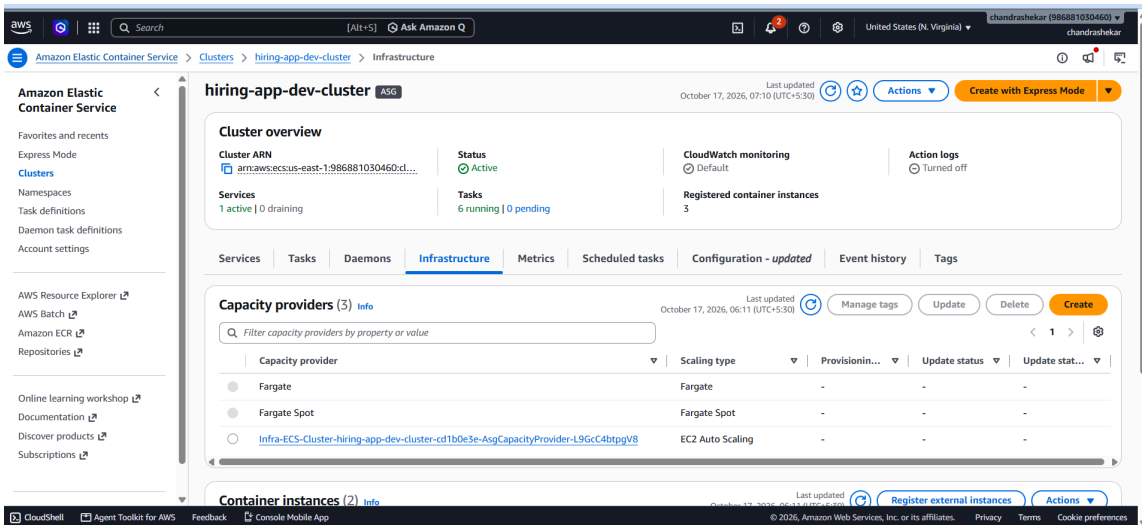
Q12. Why does the ALB use port 80 while the container uses port 8080?

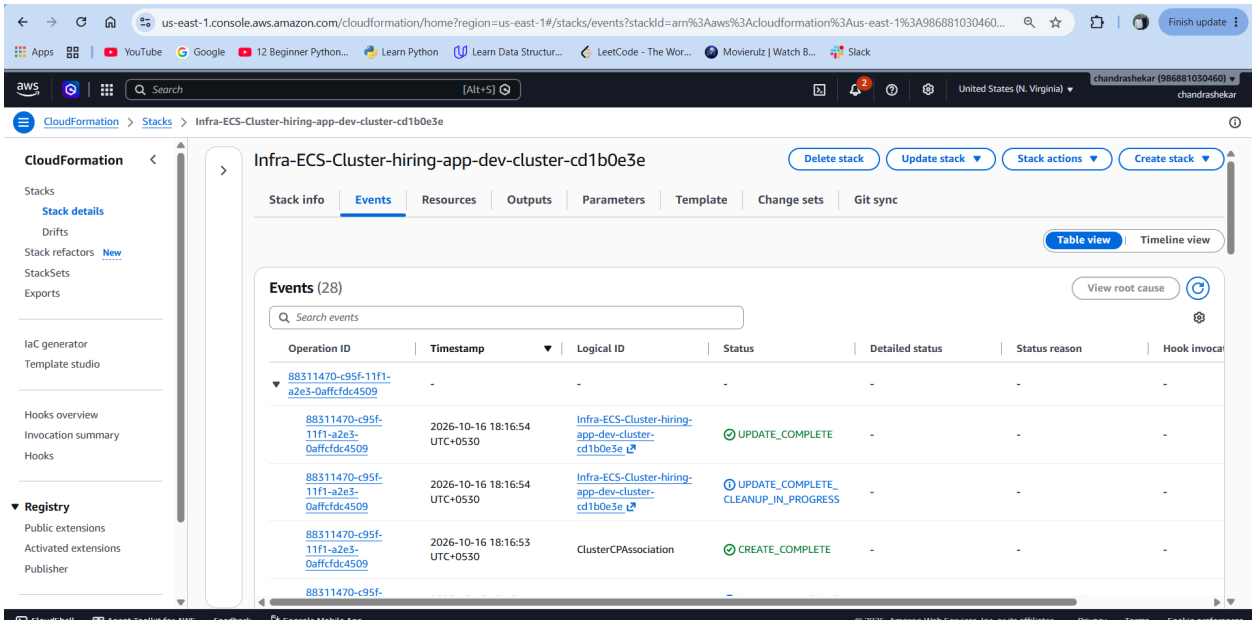
The ALB receives user traffic on port 80 and forwards it to the targets using their registered dynamic host ports. Docker maps those host ports to container port 8080.

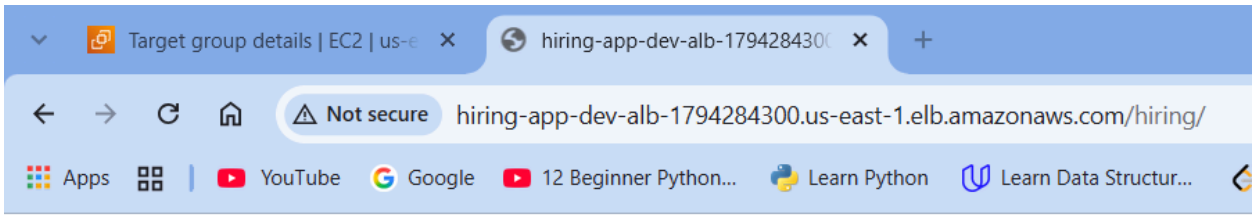



us-east-1.console.aws.amazon.com/ec2/home?region=us-east-1#Instances:v=3;$case=tags:true%5C,client:false;$regex=tags:false%5C,client:false
EC2 > Instances
Instances (5) Info
Last updated less than a minute ago
Connect
Instance state
Actions
Launch instances
Saved filter sets
Choose filter set
Filter Instance by attribute or tag (case-sensitive)
Name
Instance ID
Instance state
Instance type
Status check
Availability Zone
Public IPv4 DNS
ECS Instance - hiring-app-dev-cluster i-023b63bd82aca0f3a Running c7i-flex.large 3/3 checks passed us-east-1b ec2-44-202-67-229.co... 44.202.6
ECS Instance - hiring-app-dev-cluster i-04f8251cd8f40cd99 Running c7i-flex.large 3/3 checks passed us-east-1b ec2-3-92-206-235.com... 3.92.206.
Docker-01-Server i-0a28aceca9ceddc75 Running c7i-flex.large 3/3 checks passed us-east-1a ec2-13-223-248-228.co... 13.223.2
Docker-Challenge-Terraform-Jenkins i-09f7a437cdd488896 Stopped c7i-flex.large - us-east-1a - -
ECS Instance - hiring-app-dev-cluster i-019883ab76c5dd967 Running c7i-flex.large 3/3 checks passed us-east-1a ec2-3-239-171-184.co... 3.239.17
All instances
Instance alarm states
In alarm
None Show instances in alarm
Service health
EC2 service status
Operating normally
Health events affecting EC2 instances
Open events
None Show instances

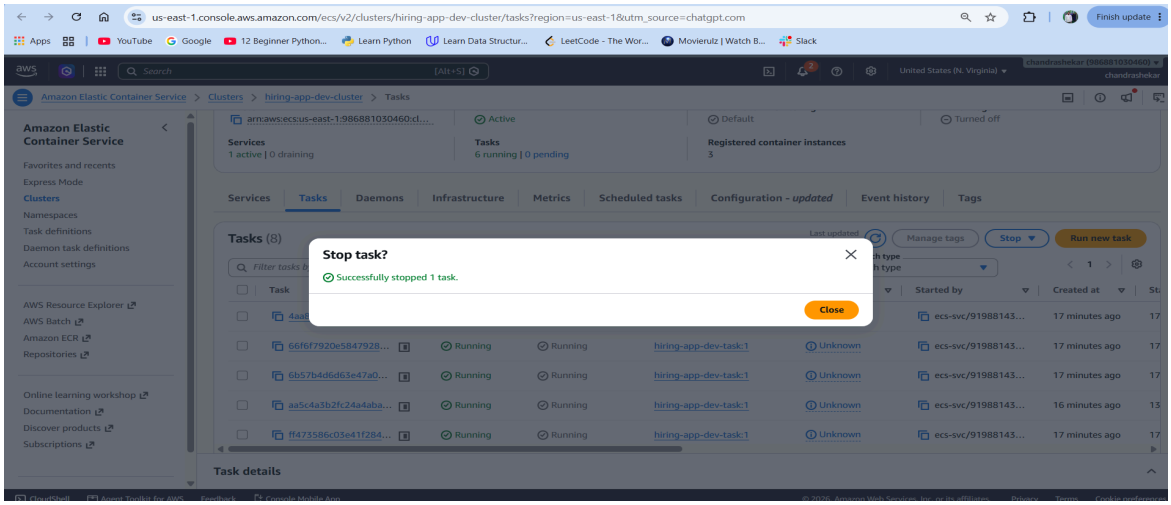

EC2 > Auto Scaling groups > Infra-ECS-Cluster-hiring-app-dev-cluster-cd1b0e3e-ECSAutoScalingGroup-UKbXQt6W29Fc
Infra-ECS-Cluster-hiring-app-dev-cluster-cd1b0e3e-ECSAutoScalingGroup-UKbXQt6W29Fc
Infra-ECS-Cluster-hiring-app-dev-cluster-cd1b0e3e-ECSAutoScalingGroup-UKbXQt6W29Fc Capacity overview
Edit
Desired capacity
3
Scaling limits
2 - 6
Desired capacity type
Units (number of instances)
Status
At desired capacity
Date created
Fri Oct 16 2026 18:16:22 GMT+0530 (India Standard Time)
Details
Integrations
Automatic scaling
Instance management
Instance refresh
Activity
Monitoring
Tags
Launch template
Launch template
lt-0f6a4f8ab8c61c21d
ECSLaunchTemplate_KhOC7aH6m04M
AMI ID
ami-07d4544a1fdee86a1
Instance type
c7i-flex.large
Owner
arn:aws:iam::986881030460:root
Version
3
Security groups
-
Security group IDs
sg-0906a6c96ccf4834f
Create time
Fri Oct 16 2026 17:51:51 GMT+0530 (India Standard Time)
Description
Storage (volumes)
Key pair name
Request Spot Instances

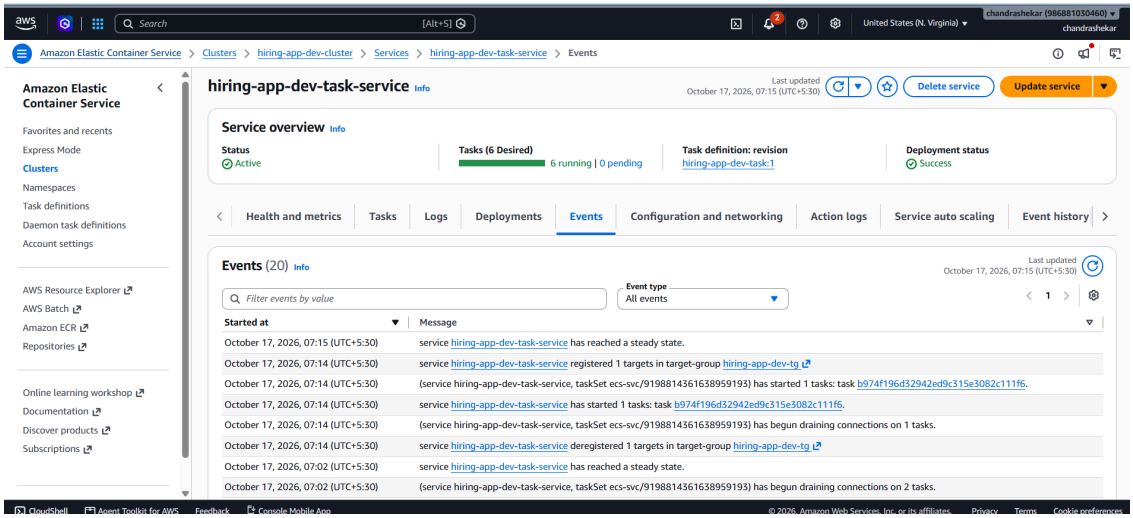

Amazon Elastic Container Service > Clusters > hiring-app-dev-cluster > Infrastructure
hiring-app-dev-cluster ASG
Last updated October 17, 2026, 07:10 (UTC+5:30)
Actions
Create with Express Mode
Cluster overview
Cluster ARN
arn:aws:ecs:us-east-1:986881030460:cl...
Status
Active
CloudWatch monitoring
Default
Action logs
Turned off
Services
1 active | 0 draining
Tasks
6 running | 0 pending
Registered container instances
3
Services
Tasks
Daemons
Infrastructure
Metrics
Scheduled tasks
Configuration - updated
Event history
Tags
Capacity providers (3) Info
Last updated October 17, 2026, 06:11 (UTC+5:30)
Manage tags
Update
Delete
Create
Filter capacity providers by property or value
Capacity provider
Scaling type
Provisionin...
Update status
Update stat...
Fargate
Fargate
Fargate Spot
Fargate Spot
Infra-ECS-Cluster-hiring-app-dev-cluster-cd1b0e3e-AsgCapacityProvider-L9GcC4btpgV8
EC2 Auto Scaling
Container instances (2) Info
Register external instances
Actions





Welcome to Techie Horizon




Search
[Alt+S]
United States (N. Virginia)
chandrashekar (986881030460)
chandrashekar
Amazon Elastic Container Service
Clusters
hiring-app-dev-cluster
Services
hiring-app-dev-task-service
Events
Amazon Elastic Container Service
Favorites and recents
Express Mode
Clusters
Namespaces
Task definitions
Daemon task definitions
Account settings
AWS Resource Explorer
AWS Batch
Amazon ECR
Repositories
Online learning workshop
Documentation
Discover products
Subscriptions
hiring-app-dev-task-service Info
Last updated
October 17, 2026, 07:15 (UTC+5:30)
Delete service
Update service
Service overview Info
Status
Active
Tasks (6 Desired)
6 running | 0 pending
Task definition: revision
hiring-app-dev-task:1
Deployment status
Success
Health and metrics
Tasks
Logs
Deployments
Events
Configuration and networking
Action logs
Service auto scaling
Event history
Events (20) Info
Last updated
October 17, 2026, 07:15 (UTC+5:30)
Filter events by value
Event type
All events
1
Started at
Message
October 17, 2026, 07:15 (UTC+5:30)
service hiring-app-dev-task-service has reached a steady state.
October 17, 2026, 07:14 (UTC+5:30)
service hiring-app-dev-task-service registered 1 targets in target-group hiring-app-dev-tg
October 17, 2026, 07:14 (UTC+5:30)
(service hiring-app-dev-task-service, taskSet ecs-svc/9198814361638959193) has started 1 tasks: task b974f196d32942ed9c315e3082c111f6.
October 17, 2026, 07:14 (UTC+5:30)
service hiring-app-dev-task-service has started 1 tasks: task b974f196d32942ed9c315e3082c111f6.
October 17, 2026, 07:14 (UTC+5:30)
(service hiring-app-dev-task-service, taskSet ecs-svc/9198814361638959193) has begun draining connections on 1 tasks.
October 17, 2026, 07:14 (UTC+5:30)
service hiring-app-dev-task-service deregistered 1 targets in target-group hiring-app-dev-tg
October 17, 2026, 07:02 (UTC+5:30)
service hiring-app-dev-task-service has reached a steady state.
October 17, 2026, 07:02 (UTC+5:30)
(service hiring-app-dev-task-service, taskSet ecs-svc/9198814361638959193) has begun draining connections on 2 tasks.
CloudShell
Agent Toolkit for AWS
Feedback
Console Mobile App
© 2026, Amazon Web Services, Inc. or its affiliates.
Privacy
Terms
Cookie preferences